BigQuery DDL & YAML Generator Tool – Usage Guide

Table of Contents

- Table of Contents
  - Overview
  - How to Use
    - 1. Add tables to the input file
      - Sample table input
    - 2. Run in dry-run mode (recommended first)
    - 3. Manually verify output
    - 4. Run actual execution
    - 5. Validate before commit
    - 6. Final verification
    - 7. Clean up tables.txt
    - 4. Run actual execution
    - 5. Validate before commit
    - 6. Final verification
    - 7. Clean up tables.txt
    - 8. Commit to Git
- Terrafrom POC:
  - Removing Resources from Terraform (BigQuery Tables)
  - Method 1 — Using removed {} Block (Declarative)
    - Example
    - What this does
    - Important Limitation
  - Method 2 — Using state rm Command (Imperative)
    - Example (Terragrunt)
    - What this does
  - When to Use Which
  - Summary

Overview

This utility automates generating BigQuery deployment artifacts from a simple table list.

For each input table, it produces:

- **V1 SQL** → `CREATE TABLE` statements (schema, partitioning, clustering)
- **V2/V3 SQL** → Policy tag `ALTER` statements (if applicable)
- `clustering.yaml` → Clustering metadata (if clustering exists)
- **DAG Entry** → Updates central `bigquery_table_management_config.yaml` for orchestration

It is designed to be:

- **Safe** → Supports dry-run mode to preview changes without writing files
- **Repeatable** → Runs per table or for multiple tables
- **Structured** → Outputs organized by dataset/table in predictable directories
- **Team-friendly** → Uses a simple text input file (`tables.txt`) shared by teams

The workflow is:

1. Add tables to `tables.txt`
2. Run script in **dry-run** mode to preview changes
3. Manually validate SQL, YAML, and DAG entries
4. Run script in **actual execution** mode
5. Run validation tests
6. Perform final review and ownership checks
7. Clean up `tables.txt`
8. Commit and push changes

How to Use

1. Add tables to the input file

Update `dags/bigquery_table_management/precursor/tables.txt` with the tables to process.

Each line follows:

```
project.dataset.table
```

This file is the **single source of truth** for which tables the utility generates artifacts.

Sample table input

```
pcb-uat-landing.cots_hr.NEW_HIRE
pcb-uat-landing.domain_cdic.PCMA_CDIC_0201
pcb-uat-landing.domain_cdic.CDIC_RESPONSIBLE_PARTY
```

Notes:

- One table per line
- No commas or extra delimiters
- Lines starting with `#` are comments and ignored
- Group related tables logically (e.g., by dataset or domain) for easier review

2. Run in dry-run mode (recommended first)

```python
if __name__ == "__main__":

    # Each line should contain one table:
    # project.dataset.table
    # Ignores empty lines and comments (#).

    tables = []
    file_path = r"dags/bigquery_table_management/precursor/tables.txt"
    with open(file_path, "r") as f:
        for line in f:
            line = line.strip()
            # skip empty lines and comments
            if not line or line.startswith("#"):
                continue
            tables.append(line)

    generator = BigQueryDDLGenerator(
        output_dir=r"dags/bigquery_table_management/schemas/"
    )

    # for running in the dry run mode
    generator.process_tables(tables, dry_run=True)


    # # for actually generating he sql files
    # generator.process_tables(tables)
```

Always validate changes using **dry-run** mode first. This shows what will be created or modified.

In code, invoke dry-run as:

```python
# generator.process_tables(tables)
generator.process_tables(tables, dry_run=True)
```

Run the script from bash:

```bash
# Run precursor script in dry-run mode
python dags/bigquery_table_management/bq_table_management_precursor.py
```

This will:

- Print generated SQL (V1 + V2/V3) to console
- Print YAML content (V2 YAML artifacts)
- Show target file paths for all artifacts

  → No files are created or modified in dry-run mode

Use this safe preview to catch issues (wrong project/dataset, missing policy tags) before writing files.

3. Manually verify output

```yaml
sk_test_sales_table_management:
  default_args:
    owner: ''
  table: pcb-poc-datalake1.sk_test.sales
  scripts:
    - sk_test/sales/V1__CREATE_TABLE.sql

sk_test_sales_new_table_management:
  default_args:
    owner: ''
  table: pcb-poc-datalake1.sk_test.sales_new
  scripts:
    - sk_test/sales_new/V1__CREATE_TABLE.sql
    - sk_test/sales_new/V2__CLUSTER_BY.yaml
    - sk_test/sales_new/V3__ALTER_TABLE.sql

domain_account_management_ACCOUNT_table_management:
  default_args:
    owner: ''
  table: pcb-uat-landing.domain_account_management.ACCOUNT
  scripts:
    - domain_account_management/ACCOUNT/V1__CREATE_TABLE.sql

cots_hr_NEW_HIRE_table_management:
  default_args:
    owner: ''
  table: pcb-uat-landing.cots_hr.NEW_HIRE
  scripts:
    - cots_hr/NEW_HIRE/V1__CREATE_TABLE.sql
    - cots_hr/NEW_HIRE/V2__ALTER_TABLE.sql
```

After dry-run, review console output carefully.

Focus on:

- **SQL correctness**
  - Check table names, datasets, projects
  - Validate column definitions, data types, modes
  - Verify partitioning and clustering definitions
  - Confirm policy tags apply as expected
- **YAML formatting and content**
  - Ensure correct, consistent indentation
  - Confirm required keys present
  - Verify naming follows project standards
- **DAG entries in** `bigquery_table_management_config.yaml`
  - Verify table identifiers and dataset references
  - Check scheduling, dependencies, flags
  - Confirm new entries don't conflict with existing configs

If incorrect, **fix schema or config** then re-run dry-run. Repeat until output is correct.

4. Run actual execution

Once dry-run output is correct, proceed to file generation.

Remove the dry-run flag:

```
generator.process_tables(tables)
# generator.process_tables(tables, dry_run=True)
```

Run the script again:

```
# Run precursor script in execution mode
python dags/bigquery_table_management/bq_table_management_precursor.py
```

In this mode, the utility will:

- Create or update **V1 SQL** files for specified tables
- Create or update **V2/V3 SQL** for policy tag changes (if applicable)
- Generate or update corresponding **YAML** artifacts
- Update relevant **DAG configuration files** (e.g., `bigquery_table_management_config.yaml` / `dag.yaml`)

After execution, check:

- **SQL files**
  - Confirm files exist in expected directories
  - Verify contents match dry-run output
- **YAML files**
  - Validate structure, required fields, naming
- **DAG entries** (e.g., in `dag.yaml` or equivalent)
  - Confirm new entries are present and correctly wired

Proceed only when confident generated artifacts are correct.

5. Validate before commit

```
(airflow-venv) AMAML0BKGMFC3:pcb-data-foundation-airflow-dags suprabhat.kumar$ python dags/bigquery_table_management/bq_table_management_precursor_validation.py

🔍 Running table validation tests...

========================================================================================== test session starts ==========================================================================================
platform darwin -- Python 3.12.11, pytest-9.0.1, pluggy-1.6.0
rootdir: /Users/suprabhat.kumar/Desktop/GitProjects/pcb-data-foundation-airflow-dags
configfile: pytest.ini
plugins: anyio-4.12.1, cov-6.0.0
collected 1 item

../tests/dags/bigquery_table_management/test_pr_sql_validation.py F                                                                                                                                  [100%]

================================================================================================= FAILURES =================================================================================================
_____________________________________________________________________________________________ test_pr_sql_validations _____________________________________________________________________________________________

    def test_pr_sql_validations():
        """
        Main PR validation test entry point.

        Scans all files under dags/bigquery_table_management/schemas/ (no git).
        Only .gitkeep and README.md are ignored. All other files must pass naming
        validation (V<n>__OPERATION.sql or .yaml) or validation fails.
        Runs in CI and locally.
        """

        if not SCHEMA_ROOT.exists():
            logger.warning(
                "Schema root does not exist: %s. Skipping validation.", SCHEMA_ROOT
```

Before committing, run validation tests to ensure artifacts are consistent and pass quality checks.

Use this command:

```
python
dags/bigquery_table_management/bq_table_management_precursor_validation
.py
```

Ensure:

- All tests pass with **no failures**
- No unexpected or unintended changes (e.g., unrelated files modified, large diffs)

If validation fails, fix issues or revert changes, then rerun until clean.

6. Final verification

Perform a final manual review focusing on ownership and consistency.

- Review `bigquery_table_management_config.yaml` manually
  - Confirm each new table entry is present and correct
- Add or update owner details:
  - Set `owner: ''` to appropriate team or individual
  - Use consistent owner naming conventions
- Ensure:
  - No duplicate table entries
  - No incorrect or conflicting configurations
  - No accidental changes to unrelated sections
- Confirm overall file structure and order remain consistent
  (e.g., sorting, grouping by dataset/domain, indentation style)

This step reduces risk of hidden config issues appearing in production.

7. Clean up `tables.txt`

After successful run and validation:

- Remove processed table entries from:
  - `dags/bigquery_table_management/precursor/tables.txt`
- Keep `tables.txt` clean and minimal
  - Free from stale or processed tables
  - Avoid duplicate entries
- Avoid:
  - Leaving duplicate entries

    - Keeping historical entries already deployed tags are applied as expected
- **YAML formatting and content**
    - Ensure indentation is correct and consistent
    - Confirm all required keys are present
    - Verify that naming conventions follow the project standards
- **DAG entries in** `bigquery_table_management_config.yaml`
    - Verify table identifiers and dataset references
    - Check that scheduling, dependencies, and flags are correct
    - Confirm the new entries do not conflict with existing configurations

If anything looks incorrect, **fix the underlying schema or configuration** first, then re-run the dry-run. Iterate until the preview output is fully correct.

4. Run actual execution

Once everything looks correct in dry-run mode, you can proceed with actual file generation.

Update your invocation to remove the dry-run flag:

```
generator.process_tables(tables)
# generator.process_tables(tables, dry_run=True)
```

Then execute the script again:

```
# Run the precursor script in actual execution mode
python dags/bigquery_table_management/bq_table_management_precursor.py
```

In this mode, the utility will:

- Create or update the **V1 SQL** files for the specified tables
- Create or update **V2/V3 SQL** for policy tag changes (if applicable)
- Generate or update the corresponding **YAML** artifacts
- Update the relevant **DAG configuration files** (e.g. `bigquery_table_management_config.yaml` / `dag.yaml` as applicable)

After execution, carefully check:

- **SQL files**
    - Confirm files exist in the expected directories
    - Verify the contents match what you saw in the dry-run
- **YAML files**
    - Validate structure, required fields, and naming

- **DAG entries** (for example, in `dag.yaml` or equivalent)
  - Confirm the new entries are present and correctly wired

Only proceed once you are confident the generated artifacts are correct.

5. Validate before commit

Before committing any changes, run the validation tests to ensure that the generated artifacts are consistent and pass basic quality checks.

Use the command below:

```
python
dags/bigquery_table_management/bq_table_management_precursor_validation
.py
```

Ensure:

- All tests pass with **no failures**
- There are **no unexpected or unintended changes** (e.g. unrelated files modified, large diffs in existing configs)

If the validation fails, fix the reported issues (or revert problematic changes), then rerun the validation until it passes cleanly.

6. Final verification

Perform a final manual review of the configurations, focusing on ownership and consistency.

- Review `bigquery_table_management_config.yaml` manually
  - Confirm each new table entry is present and correctly configured
- Add or update owner details:
  - Set the `owner: ''` field to the appropriate team or individual
  - Use consistent owner naming conventions used elsewhere in the file
- Ensure there are:
  - No duplicate table entries
  - No incorrect or conflicting configurations

  - No accidental changes to unrelated sections
- Confirm the overall file structure and ordering remain consistent (e.g. sorting, grouping by dataset/domain, indentation style)

This step reduces the risk of hidden config issues surfacing later in production.

7. Clean up `tables.txt`

After a successful run and validation:

- Remove the processed table entries from:
  - `dags/bigquery_table_management/precursor/tables.txt`
- Keep `tables.txt`:
  - Clean and minimal
  - Free from stale or already-processed tables
- Avoid:
  - Leaving duplicate entries
  - Keeping historical entries that have already been deployed

A clean `tables.txt` helps other teams use the same mechanism without confusion and prevents accidental re-generation of artifacts for existing tables.

8. Commit to Git

Once all checks and validations are complete, commit your changes.

From your repository root, run:

```
git add .
git commit -m "Added DDL and YAML for new tables"
git push
```

Guidelines:

- Use a clear commit message that describes the scope (e.g. domain, dataset, or business feature)
- If you are adding many tables, consider referencing a ticket/issue ID in the commit message
- Optionally create a short summary in your PR description, listing the new tables and their purpose

---

Terrafrom POC:

Removing Resources from Terraform (BigQuery Tables)

To stop managing a resource in Terraform, use one of two supported approaches. The correct method depends on whether you want to remove all instances of a resource or only specific ones.

Before starting, find the table variable by running:

```
terragrunt state list
```

This returns the list of resources managed by Terraform.

```
(airflow-venv) AMAMLQ0K6MFC3:terraform suprabhat.kumar$ terragrunt state list
17:11:58.738 INFO   Downloading Terraform configurations from ./modules into ./.terragrunt-cache/YEQSrjVT_FkWzuVkkNdu1L1OOkw/YCrAdpzgdWiOnT6i1ErLAhU7G8w
17:11:58.832 INFO   terraform: Initializing the backend...
17:11:58.986 INFO   terraform: Initializing provider plugins...
17:11:58.986 INFO   terraform: - Reusing previous version of hashicorp/google from the dependency lock file
17:11:59.590 INFO   terraform: - Using previously-installed hashicorp/google v7.26.0
17:11:59.590 INFO   terraform: Terraform has been successfully initialized!
17:11:59.590 INFO   terraform:
17:11:59.590 INFO   terraform: You may now begin working with Terraform. Try running "terraform plan" to see
17:11:59.590 INFO   terraform: any changes that are required for your infrastructure. All Terraform commands
17:11:59.590 INFO   terraform: should now work.
17:11:59.590 INFO   terraform: If you ever set or change modules or backend configuration for Terraform,
17:11:59.590 INFO   terraform: rerun this command to reinitialize your working directory. If you forget, other
17:11:59.590 INFO   terraform: commands will detect it and remind you to do so if necessary.
google_bigquery_table.tables["tf_v2"]
google_bigquery_table.tables["tf_v3"]
google_bigquery_table.tables["tf_v4"]
google_bigquery_table.tables["tf_v5"]
(airflow-venv) AMAMLQ0K6MFC3:terraform suprabhat.kumar$
```

Method 1 — Using `removed {}` Block (Declarative)

The `removed` block is a Terraform-native, declarative way to remove a resource from state without deleting the actual infrastructure.

Example

```
removed {
  from = google_bigquery_table.tables

  lifecycle {
    destroy = false
  }
}
```

What this does

- Removes the resource from Terraform state
- Keeps the actual BigQuery tables intact
- Prevents accidental deletion

Important Limitation

- Works only at the resource level
- Removes all instances created via `for_each`

Example:

```
resource "google_bigquery_table" "tables" {
```

```
  for_each = var.tables
}
```

Using `removed {}` here removes:

```
google_bigquery_table.tables["m_tf_v1"]
google_bigquery_table.tables["m_tf_v2"]
google_bigquery_table.tables["m_tf_v3"]
...
```

You cannot remove individual keys like `"tf_v1"` using this method.

```
Apply complete! Resources: 4 added, 0 changed, 0 destroyed.
(airflow-venv) AMAMLQ0K6MFC3:tables suprabhat.kumar$ terraform plan

  Error: Resource instance keys not allowed

    on removed.tf line 2, in removed:
     2:   from = google_bigquery_table.tables["m_tf_v1"]

  Resource address must be a resource (e.g. "test_instance.foo"), not a resource instance (e.g. "test_instance.foo[1]").

(airflow-venv) AMAMLQ0K6MFC3:tables suprabhat.kumar$
```

Method 2 — Using `state rm` Command (Imperative)

This method allows fine-grained removal of specific resources or instances from Terraform state.

Example (Terragrunt)

```
terragrunt state rm 'google_bigquery_table.tables["m_tf_v1"]'
```

What this does

- Removes only the specified instance (`tf_v1`) from state
- Leaves the actual BigQuery table unchanged
- Keeps other tables under Terraform management

When to Use Which

Scenario	Recommended Method
Remove entire resource (all tables)	`removed {}`
Remove specific tables (subset of keys)	`state rm`

Summary

- `removed {}` → declarative, removes entire resource
- `state rm` → imperative, removes specific instances